ee2004

week12

ADC Example 1

Listing 7.10 Third example on the usage of the ADC module, the C++ code

```
#include "mbed.h"

#define WAIT_TIME_MS 100

AnalogIn intTemp(ADC_TEMP);

int main()
{
while (true)
{
int value = intTemp.read_u16();
printf("Analog value read %u \n", (unsigned int)value);

thread_sleep_for(WAIT_TIME_MS);
}
}
```

Digital to Analog Conversion in Embedded Systems

Digital to analog conversion (DAC) aims to obtain the analog representation of a given digital value. We will cover two methods to perform this operation.

The first method is called zero-order hold (ZOH) which can be performed if the embedded system has a DAC module to be used for this purpose.

The second method is based on pulse width modulation (PWM). This method does not require any specific module except the timer which should be available in most embedded systems.

Since the ZOH and PWM depend on physical hardware of the embedded system, we will consider how these operations are done in the STM32F4 microcontroller in a separate section.

Zero-Order Hold

- The zero-order hold (ZOH) operation, performed by the DAC module, works by a simple principle such that a constant voltage level is fed to output for a given digital input value.
- This voltage is kept at output for a predefined time. When another digital value comes to the DAC module, the output voltage changes accordingly.
- There should be associated pins for the module within the embedded system to handle analog voltage values.

- We can explain working principles of the DAC module with the help of the concepts introduced in the ADC module. The DAC module works in a certain voltage range determined by *Vref* + in the STM32F4 microcontroller.
- The DAC module has a 12-bit digital input. Hence, the STM32F4 microcontroller has a 12- bit DAC module. When the minimum digital value (zero) is fed to it, output of the module will be 0 V. Likewise, when the maximum digital value (2*N* - 1) is fed to the DAC module, its output will be *Vref* +- where is given in Eq. 7.1.

$$\Delta = \frac{V_{ref+}}{2^{12}} \qquad (7.1)$$

digital input values, we can calculate the analog value from the DAC module as

$$V_{out} = \Delta \times d \tag{7.4}$$

where d is the digital value fed to the DAC module. It is evident from Eq. 7.4 that we have limited analog output values since d is digital. We schematically explained these concepts in Fig. 7.15.

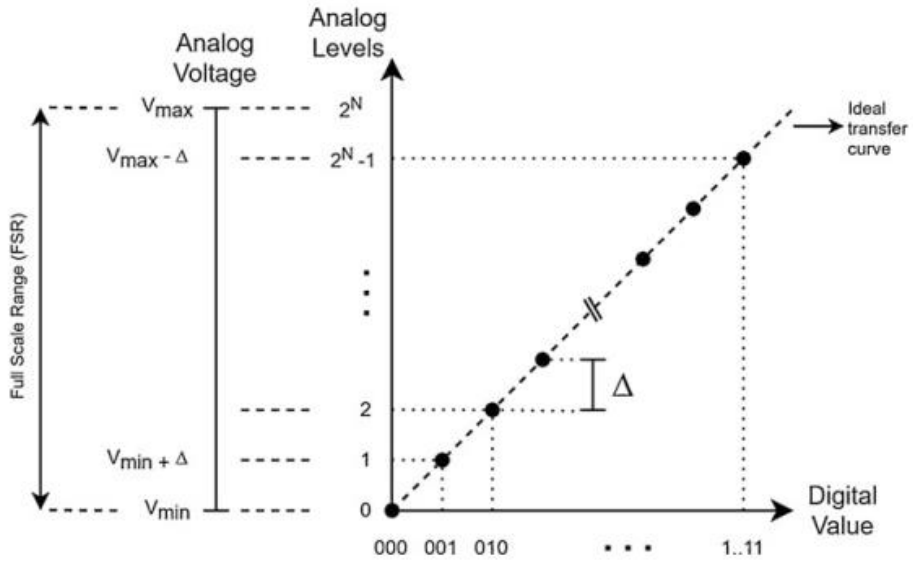


Fig. 7.15 DAC transfer function

If we feed successive digital values to the DAC module, then we will have a staircase like analog signal at output. We provide such a signal in Fig. 7.16. This signal can be smoothed out further by a low-pass filter connected to output of the DAC module.

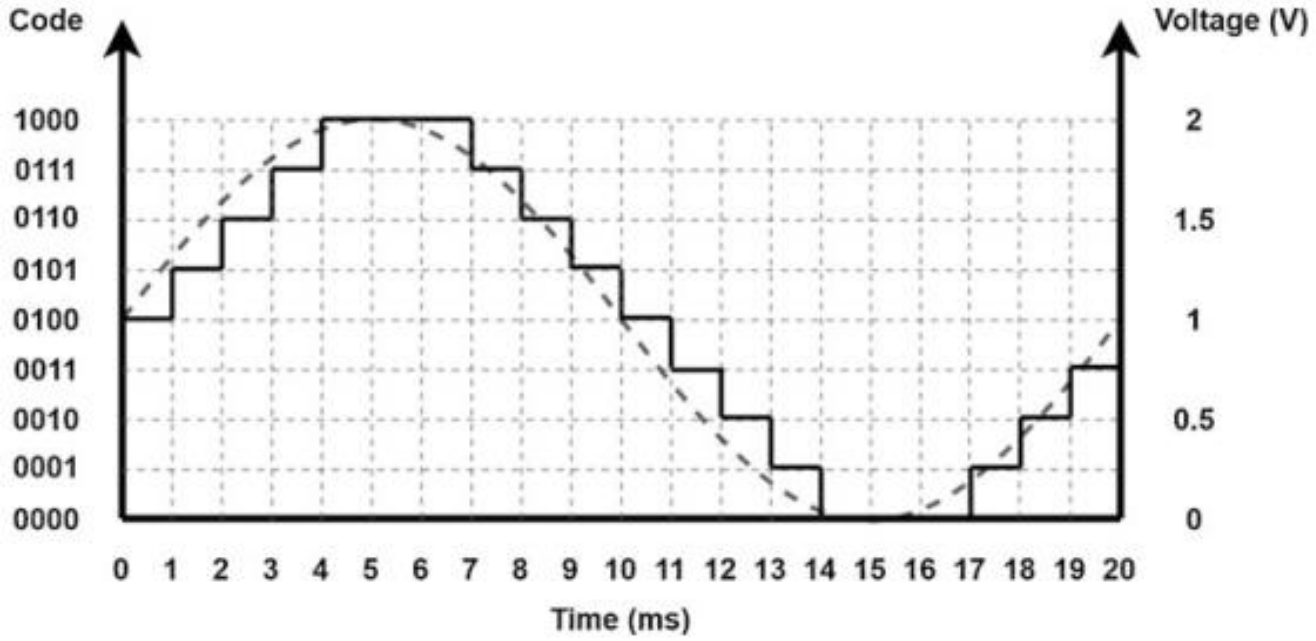


Fig. 7.16 DAC signal output example

Pulse Width Modulation

- The embedded system may not have a dedicated DAC module to convert a digital value to analog form.
- For such cases, we can benefit from pulse width modulation (PWM).
- PWM provides fixed amplitude pulses with varying width. These pulses are formed periodically such that there is a base periodic signal composed of pulses with constant amplitude, with value *VDD*, and varying width.
- The important point here is that period of the base periodic signal is low. In other words, frequency of this signal is high. When such a high-frequency signal is fed to an analog system, it cannot detect such high-frequency changes. Instead, the analog system observes the average voltage value within one period of the PWM signal.
- We can explain this process as follows. The analog system may not respond to high-frequency changes. Hence, it acts as a low-pass filter. As a result, we can change the effective voltage value observed by the analog system by changing the pulse width.

We provide three scenarios on obtaining different analog voltage values with different duty cycles in Fig. 7.17. In this figure, a small portion of PWM signals are provided. As can be seen in this figure, V_{avg} changes linearly with the selected duty cycle. The duty cycle can also change on the fly. Hence, we can have a varying analog voltage value at output.

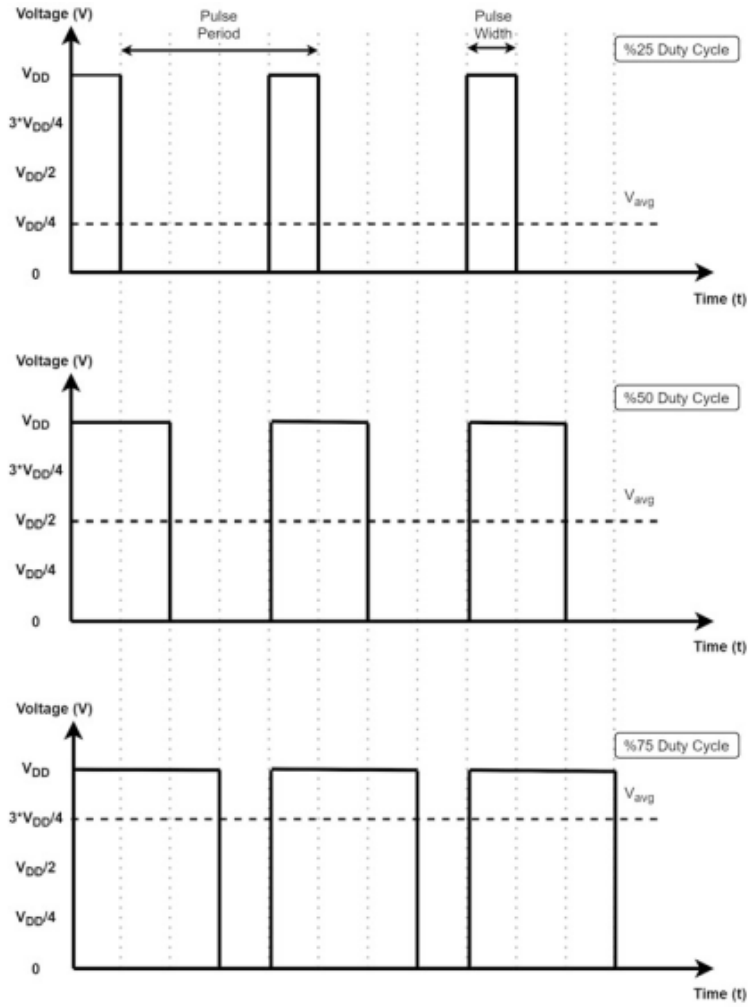


Fig. 7.17 PWM signals with different duty cycles

- We can generate three different PWM signals as left aligned, right aligned, and centered.
- Fig. 7.18. As can be seen in this figure, when the signal is left aligned, it always starts with the *VDD* value in one period. The signal then goes to 0 V after *ton* duration.
- When the signal is right aligned, it always starts with 0 V within one period. Then, it goes to *VDD* and stays there for *ton* duration till the end of period.
- Finally, we can have the centered PWM signal such that it always starts and ends with 0 V. Within the period, the signal goes to the *VDD* value and stays there with *ton* duration.

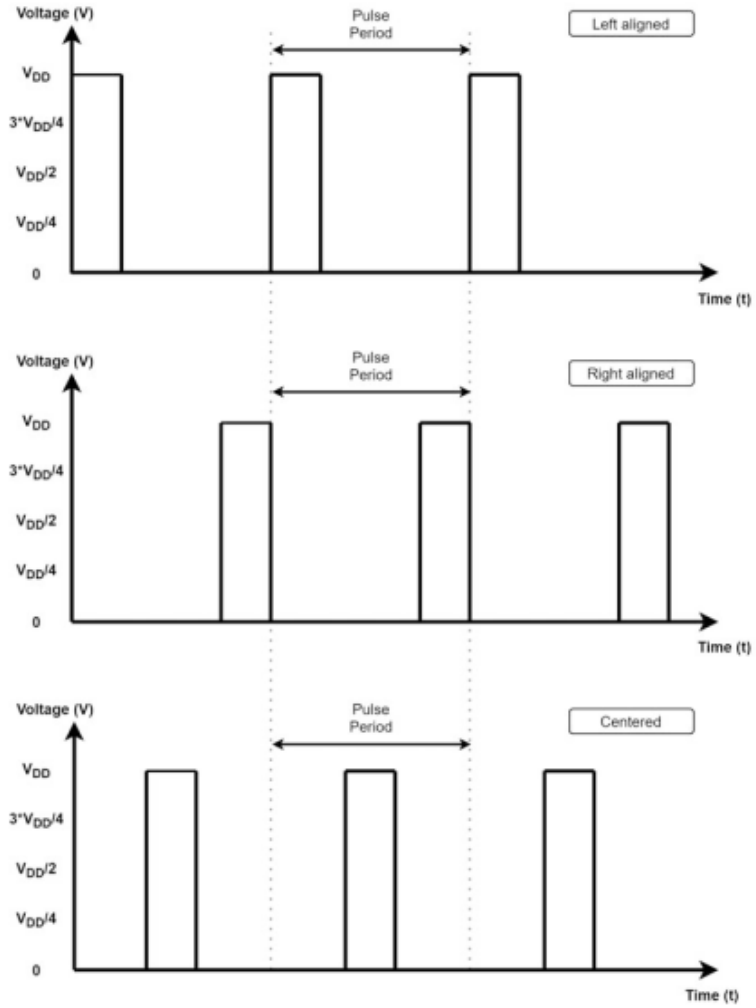


Fig. 7.18 Three different PWM alignment options

The DAC Module

- There is one 12-bit buffered voltage output DAC module in the STM32F4 microcontroller.
- General block diagram of this module is as in Fig. 7.19. As can be seen in this figure, the DAC module has two output channels each with its own digital to analog converter.
- There is an optional buffer at output of each channel. The output impedance can be reduced by enabling it. Hence, higher loads can be driven by the microcontroller without needing additional external components.
- The DAC module can be configured either to 12- or 8-bit mode. The input can be left or right aligned in the 12-bit mode.

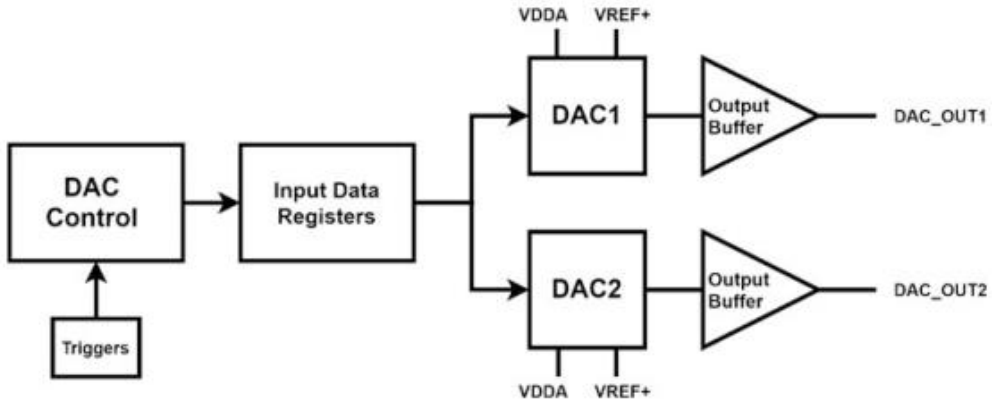


Fig. 7.19 Block diagram of the DAC module

PWM Signal Generation

The PWM signal can be generated by base timer modules of the STM32F4 microcontroller. To do so, we should use the PWM setup of these modules. In fact, the PWM signal is generated using the compare mode of the base timer.

The base timer generates the PWM signal from its appropriate output channels as follows. When the counter value in the base timer reaches 0, maximum, or compare value defined for the channel, its output can be changed. This leads to PWM signal generation based on PWM mode 1 and 2.

When the base timer is counting up, output is high, while the counter is lower than the compare value in PWM mode 1. The output is low while the counter is higher than the compare value. This is just the opposite in PWM mode 2.

Alignment	Frequency f_{PWM}	Duty cycle D
Left	$\frac{f_{Timer}}{ARRx+1}$	$\frac{CCRx+1}{ARRx+1}$
Center	$\frac{f_{Timer}}{2\times(ARRx+1)}$	$1-\frac{CCRx+1}{ARRx+1}$
Right	$\frac{f_{Timer}}{ARRx+1}$	$1-\frac{CCRx+1}{ARRx+1}$

- The PWM signal can be left or right aligned (in edge) or in centered mode.
- In the left edge aligned mode and the timer counting in upward direction, the rising edge occurs when the counter value resets to 0 and falling edge occurs when the counter value reaches the compare value.
- In the right edge aligned mode, the rising edge occurs when the counter value reaches the compare value, and falling edge occurs when the counter value reaches the maximum count. In the centered mode, timer can count only in up/down mode. Here, rising and falling edges both occur when the counter value reaches the compare value.
- In PWM mode 1 and centered mode, falling edge occurs while the counter is counting in the upward direction, and falling edge occurs while in downward direction. In addition to these modes, output polarity for the signal can be selected.
- Based on the alignment of the PWM signal, we can obtain different frequency and duty cycle values. We tabulate these in Table 7.1.

DAC Setup via C++ Language

We can benefit from Mbed and available functions there to adjust the DAC module and PWM signal properties.

In order to set up the DAC module, we should first adjust the related DAC pins. This can be done by the class AnalogOut.

We should supply a pin name we would like to use to this class. Afterward, the analog value can be set using the class AnalogOut or one of the functions write or write_u16.

The AnalogOut class and write function sets a float value between 0 and 1.0. As an example, if the reference voltage value is 3 V and we want to set the output voltage to 0.75 V, then the value within these functions will be 0.75/3 = 0.25.

The function write_u16 sets value between 0 and 0xFFFF. Here, 0xFFFF corresponds to *Vref* +. Therefore, we should use 0x8000 in the function if we want to get an output voltage *Vref* +/2.

PWM Setup via C++ Language

In order to set up the PWM signal, we should first adjust the related pins.

This can be done by the class PwmOut.

We should supply a pin name for this class. Afterward, the period of PWM signal should be set using one of the functions period, period_ms, or period_us.

These functions set the period in seconds, milliseconds, and microseconds, respectively.

Duty cycle of the PWM signal can be set using the class PwmOut or one of the functions pulsewidth, pulsewidth_ms, or pulsewidth_us.

Here, the PwmOut class and the function pulsewidth set the duty cycle in seconds.

The function pulsewidth_ms sets the duty cycle in milliseconds. The function pulsewidth_us sets the duty cycle in microseconds.

DAC Usage in C Language

- We handle the DAC usage in C language in two separate parts based on the DAC module and PWM signal generation. For both cases, we benefit from the available HAL library functions.
- The first example, with the C code given in Listing 7.14, aims to familiarize the reader with the setup and usage of the DAC module.
- Within the code, the DAC operation is triggered with software in an infinite loop. In order to execute the code in Listing 7.14, create an empty project and configure pin PA5 as DAC_OUT2.
- Afterward, enable the buffer and select the software trigger. As the project is compiled and run, brightness of the LED (connected to pin PA5) will change every second.

Listing 7.14 Usage of DAC module with software trigger, the C code

```
/* USER CODE BEGIN PV */
uint32_t i = 2048;
/* USER CODE END PV */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
i += 128;
if (i > 4095)
i = 2048;

HAL_DAC_SetValue(&hdac, DAC_CHANNEL_2, DAC_ALIGN_12B_R, i);
HAL_DAC_Start(&hdac, DAC_CHANNEL_2);
HAL_Delay(1000);
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

- We next provide an example on PWM signal generation. To do so, we should connect an external LED and resistor between the ground and PE5 pins of the STM32F4 board.
- The C code for this example is given in Listing 7.17. In order to execute the code, create an empty project.
- Then, configure pin PE5 as TIM8_CH3. Afterward, configure the timer clock to 48 MHz, set the prescaler 0, and counter period to 255.
- Hence, the generated PWM signal frequency will be 187.5 kHz. As the project is compiled and run, brightness of the LED will change every second.

Listing 7.17 PWM signal generation, the C code

```
/* USER CODE BEGIN PV */
uint32_t i = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 2 */
HAL_TIM_PWM_Start(&htim9, TIM_CHANNEL_1);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
i += 16;
if (i > 0xFF)
i = 0;

__HAL_TIM_SET_COMPARE(&htim9, TIM_CHANNEL_1, i);

HAL_Delay(1000);
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

DAC Usage in C++ Language

Listing 7.18 Analog out example, the C++ code

```
#include "mbed.h"

#define WAIT_TIME_MS 1000

AnalogOut dacOut(PA_5);
uint32_t i = 0x8000;

int main()
{
while (true)
{
i += 2048;
if (i > 0xFFFF)
i = 0x8000;
dacOut.write_u16(i);

thread_sleep_for(WAIT_TIME_MS);
}
}
```

Listing 7.19 PWM out example, the C++ code

```
#include "mbed.h"

#define WAIT_TIME_MS 1000

PwmOut pwmOut(PC_8);
uint32_t i = 0;

int main()
{
pwmOut.period_us(256);

while (true)
{
i += 16;
if (i > 0XFF)
i = 0;
pwmOut.pulsewidth_us(i);

thread_sleep_for(WAIT_TIME_MS);
}
}
```

Digital Communication

We will start with the definitions based on the transmitted data and related concepts. We should first mention that some definitions may have different meanings for different digital communication types. Hence, there is no unique definition which can be applied to all digital communication types. In order to reach a consensus and explain digital communication concepts more coherently, we pick the most reasonable ones in this section and use them as they are from this point on.

We will call the message to be transmitted as data. This can be 1 byte or 1 kilobyte. Due to the used communication type, we may not be able to transmit data at once. Hence, it should be segmented. Each segment will be called data chunk.

All digital communication types, to be explained, have their own standards while transmitting data. In general, we can call the structure, containing data and other supplementary bits, transmitted to the receiver as frame (also called packet). We will extensively use the frame definition.

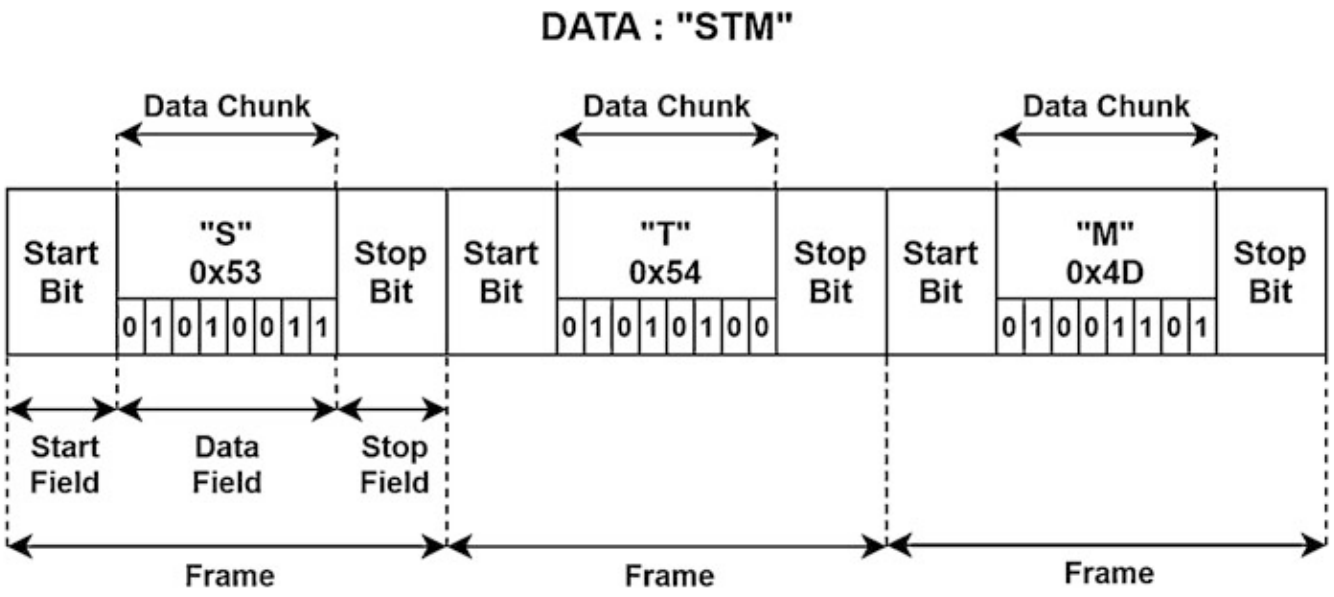


Fig. 8.1 Generic example explaining data and related concepts

- Bits and bytes within one frame are grouped. We call each group as field. As an example, I2C has start, address, read/write, data, and stop fields. A field can be formed by 1, 8, 32, or 1024 bits. This depends on the digital communication type. As a side note, a field formed by 1 bit is simply called bit. For example, instead of the start field composed of 1 bit, we will use the start bit naming instead.
- We next provide a generic example in Fig. 8.1 to explain the concepts introduced in this section. Here, we transfer the characters “STM” represented in ASCII format by a pseudo digital communication method. As can be seen in this figure, we have data, data chunks, frames, and fields.

Serial and Parallel Data Transfer

- There are two different data transfer types as parallel and serial.
- Parallel data transfer is used to achieve high transfer speeds. One such application is receiving data from a digital camera or feeding data to an LCD. The disadvantage of parallel data transfer is that it requires several pins to operate.
- Serial data transfer provides an alternative such that it only needs one or two pins. This allows communicating with more than one device at the same time. The disadvantage of serial data transfer is that it is slow since data should be transmitted sequentially.
- Fig. 8.2., we transfer 8 bits in serial and parallel form.

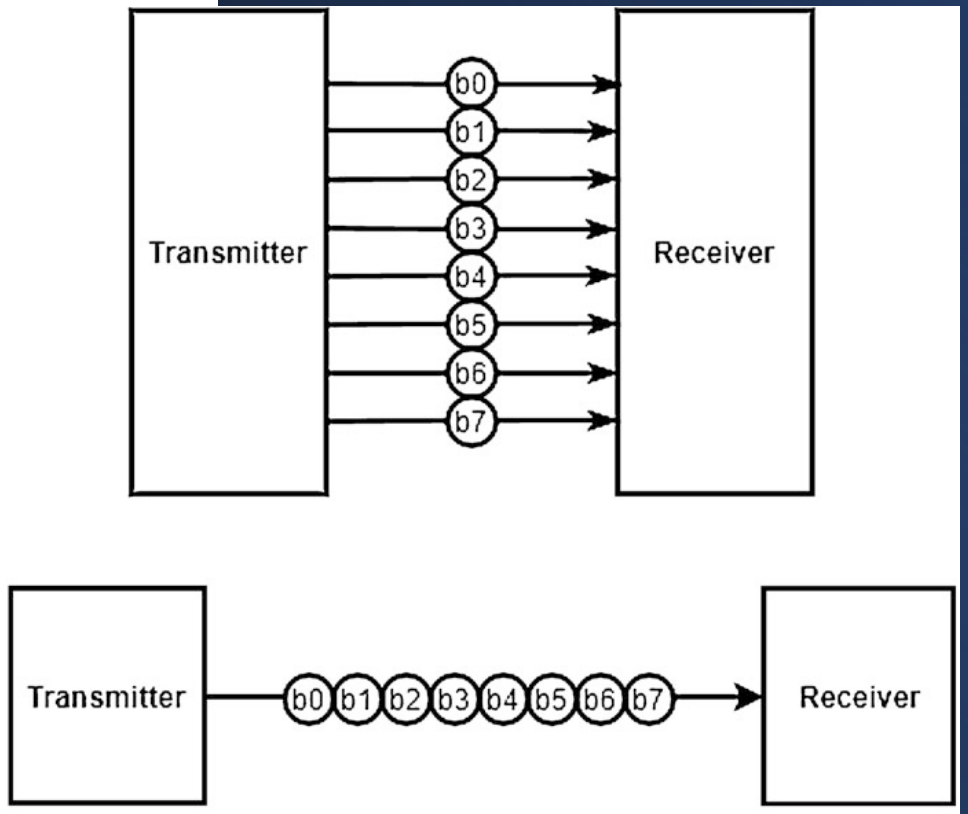
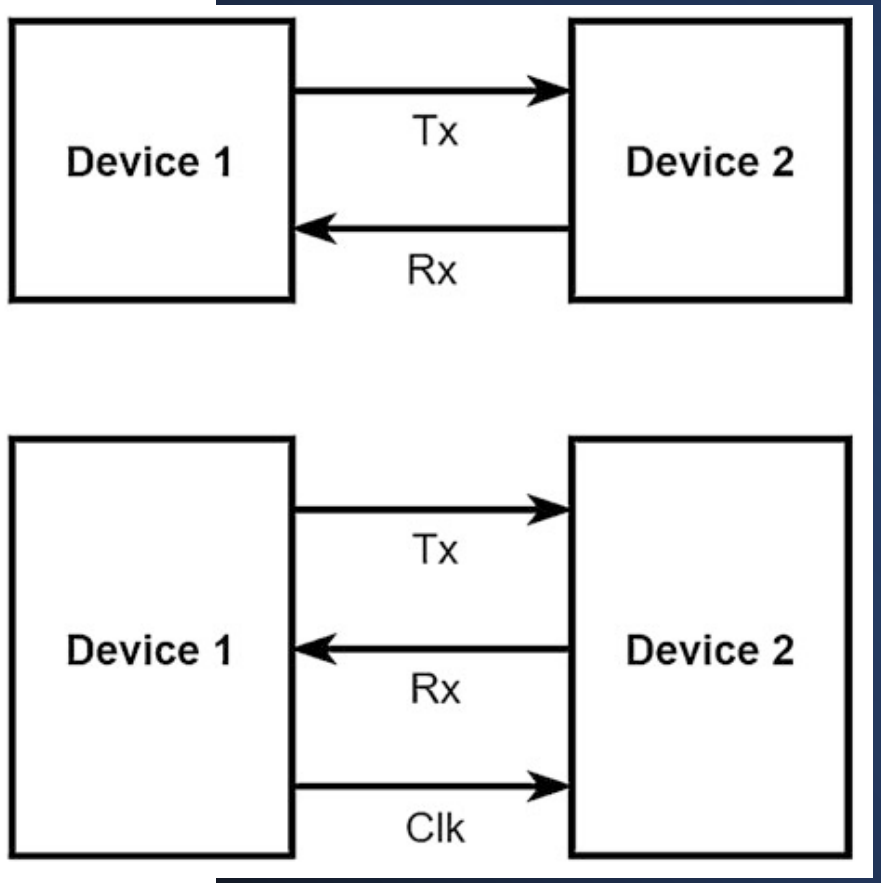


Fig. 8.2 Parallel and serial data transfer

Synchronous and Asynchronous Data Transfer

- Data transfer in digital communication can be synchronous or asynchronous.
- In synchronous data transfer, the transmitter and receiver are synchronized by a common clock signal. This synchronization signal can be sent along with data.
- The asynchronous data transfer does not need such a synchronization signal. Instead, start and stop bits for each data packet are sent. However, the transmitter and receiver should know and agree upon the transfer speed beforehand.
- The Clk pin carries the common clock signal between the transmitter and receiver. In the first setting, there is no Clk connection, or common clock, between both devices. Hence, asynchronous data transfer is achieved. In the second setting, there is a common clock signal carried by the Clk pin between the transmitter and receiver to form synchronous data transfer.



Signal Representation and Line Formations

- We introduced how a bit is represented in voltage levels in Chap. 4. We also represent signals to be transmitted or received as voltage levels in digital communication.
- Besides, there are two different line formations called single-ended and differential to transfer signals. Each formation has its advantages and disadvantages. Let’s summarize them next.
- The transferred signal is represented as a changing voltage value ref ground in the single-ended line.
- The advantage of this formation is that it requires less number of wires and less complex circuitry.
- The disadvantage of the single ended line is that the transferred signal is affected by noise and line length in long range.
- Therefore, it is used in short-range data transfer most of the times.
- For the STM32F4 microcontroller, voltage levels used in single-ended line signal transfer are compatible with the standards introduced in Chap. 4. Hence, they can be generated by GPIO pins of the microcontroller.

- Two different lines are used in the differential line formation. The transferred signal is represented by the difference of two opposite voltage values in these lines.
- As an example, the first and second lines get 3 V and 0 V, to represent logic level 1. The receiver takes the difference of these two signals and gets 3 V.
- Based on this voltage value, it decides that the signal represents logic level 1.
- The opposite voltage values are transmitted for logic level 0 representation. Hence, the first and second lines get 0 and 3 V, respectively. The receiver takes the difference of these two values and gets -3 V.
- Based on this voltage value, the receiver decides that this signal represents logic level 0.
- The advantage of the differential line usage is that it is not affected much by noise or range as in single-ended line. Hence, the differential line is used in long-range data transfer most of the times. The disadvantage of differential line is the need for extra line. Besides, the voltage levels used in operation exceed the ones available in the GPIO pins of the STM32F4 microcontroller. Hence, extra circuitry may be needed to use the differential line signal transfer.
- There must be a common ground between the connected devices in the single ended line. Having a common ground is not mandatory for the differential line formation. It depends on the communication type.

Data Encoding Types

- There are different data encoding types for digital communication. These can be summarized as
  - non return to zero,
  - return to zero,
  - non return to zero inverted,
  - return to zero inverted, and
  - Manchester coding.
- Each data encoding type has its usage areas. We will base our definitions on the single-ended line representation in this section.

Non Return to Zero

The first data encoding type is non return to zero (NRZ).

In this method, logic level 1 to be transmitted is represented by a signal with high voltage value such as 3 V.

The logic level 0 is represented by a signal with low voltage value such as 0 V.

The name non return to zero is given for the following reason. Assume that the data bit to be transmitted is logic level 1. Hence, the signal goes to high voltage. This value is kept till the next data bit arrives. Hence, it does not return to the old value.

If the next coming data bit is logic level 0, then the signal goes to low voltage. If the next data to be transmitted is again logic level 1, then the signal stays at high voltage.

Some sources call NRZ as NRZ-L such that the suffix L indicates the level-based operation.

Return to Zero

The second data encoding type is return to zero (RZ).

In this method, data representation has a changing voltage level within one clock period.

As an example, if the data bit to be transmitted is logic level 1, then the voltage level is high at the first half of the clock period and low at the second half of the clock period.

Therefore, the signal turns back to its initial low value after the high value. As a result, a clock-like signal is generated.

If the data bit to be transmitted is logic level 0, then the voltage level stays at low value.

This type of encoding can be obtained by logically ANDing the clock signal and data bits to be transmitted.

Non Return to Zero Inverted

The third data encoding type is non return to zero inverted (NRZI) which is in fact a subset of NRZ.

However, NRZI does not correspond to the actual inverse operation.

Instead, data is represented by an edge signal (low to high or high to low transition) instead of a voltage level in NRZI.

If the data bit to be transmitted is logic level 1, then a low to high or high to low transition occurs.

The transmission type is selected by the previous value of the signal.

As an example, if successive data bits have value logic level 1, then a low to high and high to low transition occur repeatedly.

If the data bit to be transmitted has value logic level 0, then the voltage level stays at its previous level.

In other words, if successive logic level zeros come, then the signal always stays at the same level.

Return to Zero Inverted

The fourth data encoding type is return to zero inverted (RZI).

This is also a subset of RZ such that it is the actual inverted operation for the RZ type.

In other words, if the data bit to be transmitted is logic level 0, then the signal will have high voltage value in the first half of the clock signal.

It will have the low voltage value in the second half of the clock signal.

If the data bit to be transmitted is logic level 1, then the signal stays at low voltage level.

Manchester Coding

The fifth data encoding type is Manchester coding.

In this method, data bits are represented by edges (low to high or high to low transition) as in NRZI.

If the data bit to be transmitted is logic level 1, then a high to low voltage transition occurs in the signal.

Therefore, it will be similar to the clock signal.

If the data bit to be transmitted is logic level 0, then a low to high voltage transition occurs in the signal.

Therefore, it will be similar to inverse of the clock signal.

This type of encoding can be obtained by logically XORing data bits with the clock signal.

Demonstrating Different Data Encoding Types

- We next provide an example to demonstrate all data encoding types. To do so, we picked an 8-bit data packet to be transmitted as 10100111. Based on this sequence, we provide the signals generated by the mentioned data encoding types in Fig. 8.4.

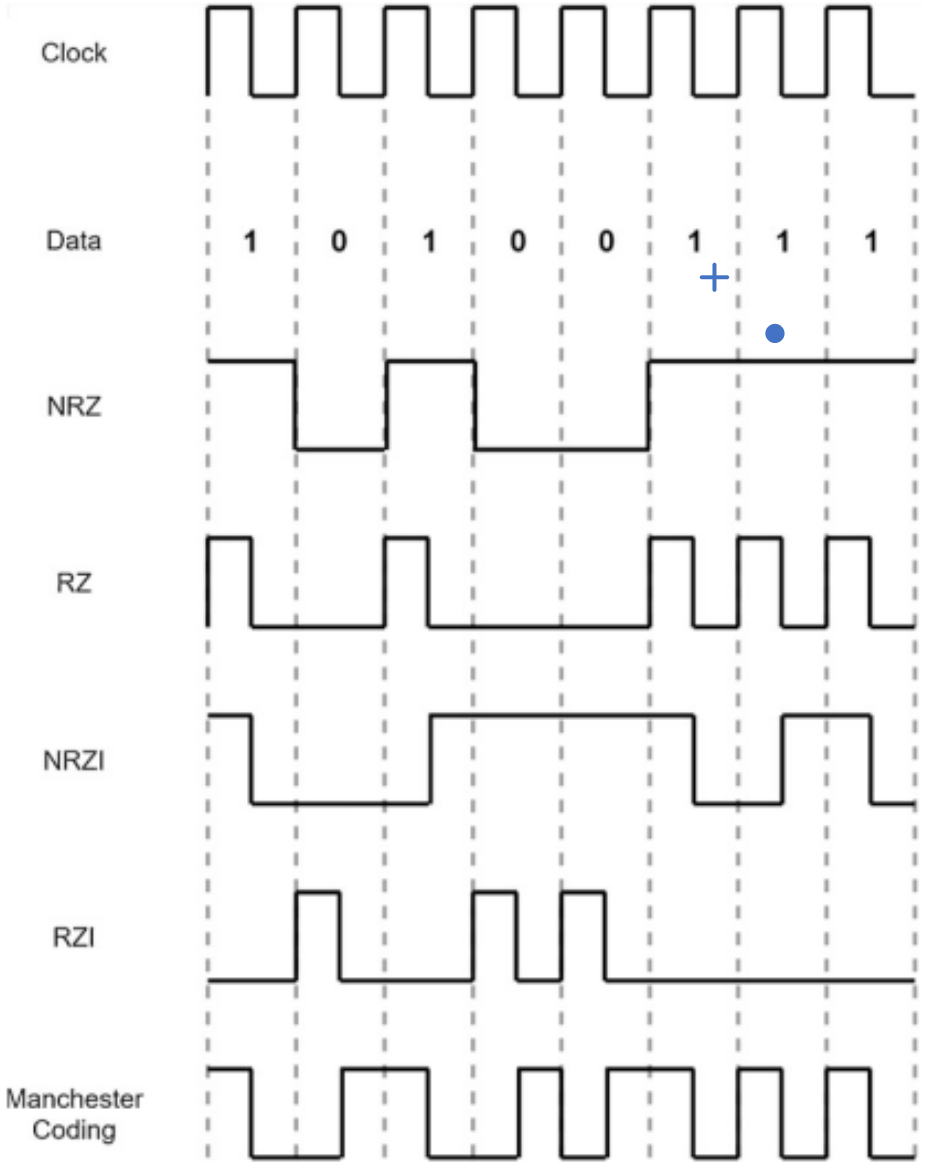


Fig. 8.4 Signals generated by different data encoding types for the given 8-bit data packet

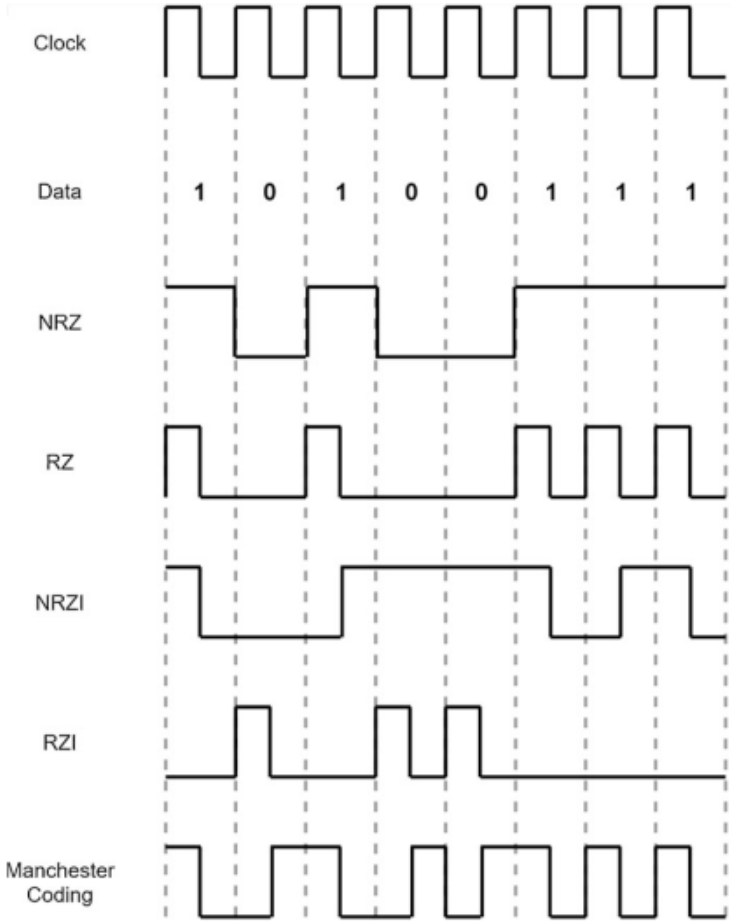


Fig. 8.4 Signals generated by different data encoding types for the given 8-bit data packet

- As can be seen in Fig. 8.4, when successive bits come as 101, the signal has high, low, and high voltage levels in NRZ data encoding.
- When the following bit values are 00, the corresponding signal stays at low voltage level. When, the following bit values are 111, the signal voltage level goes to high value and stays there.
- Next, we can observe the NRZI data encoding type. As mentioned before, the generated signal with this method is not the inverse of the one generated by NRZ.

- Instead, it has its own signal pattern for the selected 8-bit data packet. We can summarize this operation as follows.
- For the 101 part of data bits, the signal has a high to low transition in the first logic level 1 bit.
- Then, it stays at low voltage level for the logic level 0 bit. When the next logic level 1 comes, a low to high voltage transition occurs in the signal.
- As two successive logic level 0 bits come, the signal stays in high voltage level.
- When three successive logic level 1 bits come, then the signal has three successive transitions as high to low, low to high, and high to low.

- We can also analyze the signals generated by RZ data encoding type in Fig. 8.4. As can be seen in this figure, when the data bit to be transmitted is logic level 1, the signal acts as the clock signal. When a logic level 0 comes, the signal stays at low voltage level.
- The RZI data encoding type acts just in the reverse manner such that when logic level 0 comes, the signal acts as the clock signal. When logic level 1 comes, the signal stays at the low voltage level.
- Finally, Manchester coding generates a signal such that it always changes.
- To be more specific, when the first logic level 1 comes, the signal acts as the clock signal. When there is a change in successive data bits, then the signal switches from high to low or low to high value.

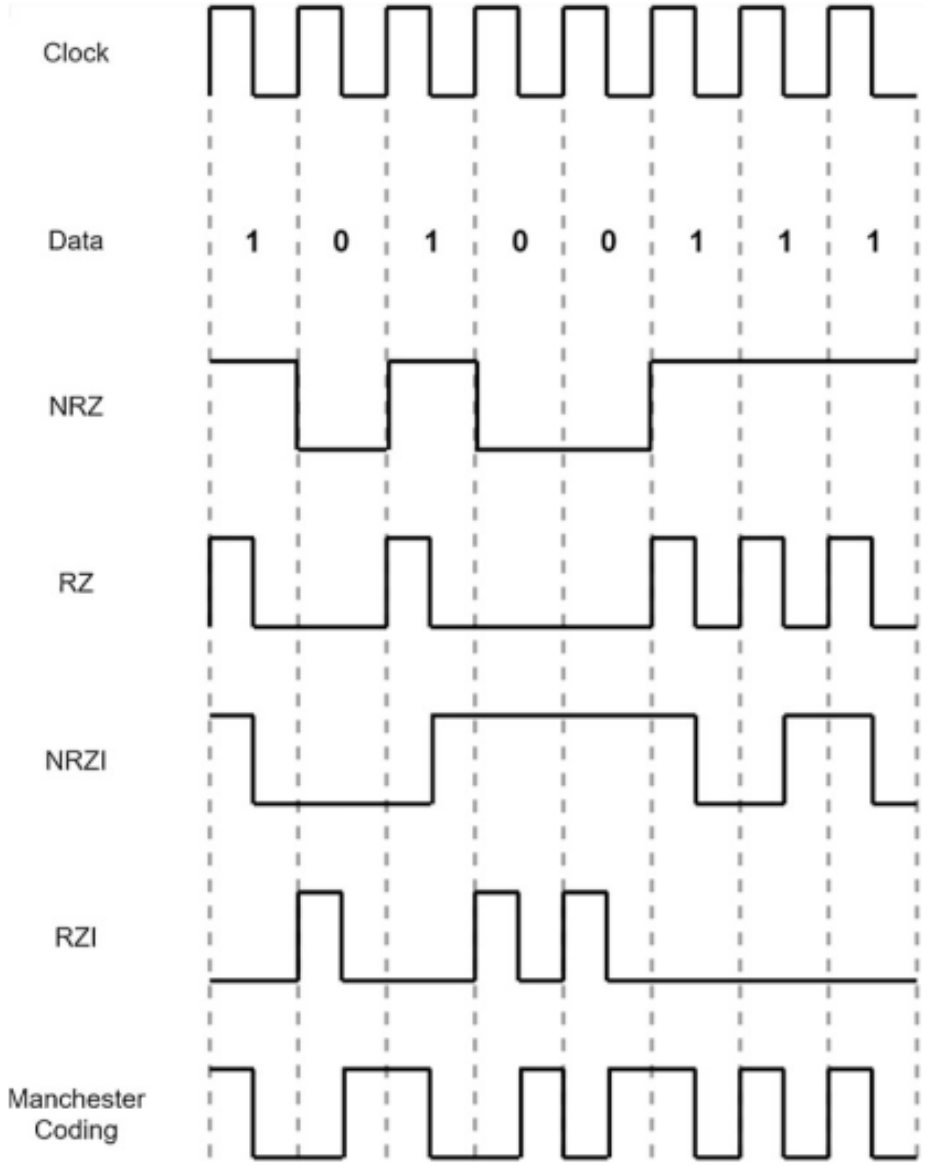


Fig. 8.4 Signals generated by different data encoding types for the given 8-bit data packet

Simplex, Half, and Full Duplex Communication

There are three communication modes as simplex, half, and full duplex to handle communication direction preferences.

The simplex mode has one-way (unidirectional) communication.

In half duplex mode, two-way (bidirectional) communication is possible. However, only one-way communication can be done at once.

In full duplex mode, two-way (bidirectional) communication is possible. Moreover, the two-way communication can be done at the same time.

As an example, serial communication can be done in either half or full duplex mode. In the former option, one pin is sufficient. In the second option, two pins are required.

Master and Slave Modes

- The next definition in digital communication is the master and slave modes.
- During data transfer between two or more devices, one can act as master and the other as slave.
- The master device generally starts the communication, asks information from slave devices, and controls them. Besides, the master device is the one generating the clock signal in synchronous communication.
- The slave device is the one responding to the master or acting in accordance with the master device commands.

Baud Rate

The baud rate indicates how many bits are transmitted in 1 second for digital systems having logic levels 0 and 1.

This also corresponds to the bit rate definition.

Both definitions indicate the speed of data transfer.

This will be extremely important in digital communication such that the higher the baud rate, the faster the communication.

We will provide specific baud rate values for different digital communication types in the following sections.

Universal Asynchronous Receiver/Transmitter

The first digital communication type we will be using is the universal synchronous/asynchronous receiver/transmitter (USART).

There is also universal asynchronous receiver/transmitter (UART).

We will be using the latter in asynchronous mode. Therefore, it will act as UART. Hence, we will only focus on UART from this point on.

As a side note, USART works in synchronous mode with an extra clock pin. In this setting, it acts as SPI in master setting.

UART Working Principles

- UART is generally preferred when fast data transfer speed is not needed.
- Hence, it is most of the times used for debugging and programming a microcontroller.
- Besides, it is used to communicate two microcontrollers.
- If two microcontrollers on different boards need to communicate with UART, a physical layer should be available such as RS232 or RS485.
- These will eliminate noise and power loss due to long-range signal transfer.

- Data transfer in UART can be done in frames.
- Each frame is composed of the start bit, data, parity bit, and stop bit(s).
- Data can be composed of 7, 8, or 9 bits.
- Start and stop bits are available in each frame. However, the parity bit can be used if needed.
- UART uses NRZ data encoding.
- In general, UART uses the TTL voltage levels to represent signals.
- UART can work in either full duplex, half duplex, or simplex modes.
- In the full duplex mode, there are two pins called Rx (receive) and Tx (transmit).
- In the half duplex mode, the Tx pin is used for both as Tx and Rx.
- In the simplex mode, Tx pin of one device is connected to Rx pin of the other device. The remaining pins are not connected.
- The master and slave definitions do not apply to UART. There is no theoretical limit for the baud rate in UART. However, a limit applies based on the used microcontroller and its associated clock.

- We next provide general setup for UART communication between two devices in Fig. 8.5. As can be seen in this figure, each device has two pins as Rx (receive) and Tx (transmit). They are connected such that one Rx pin is connected to Tx and vice versa.

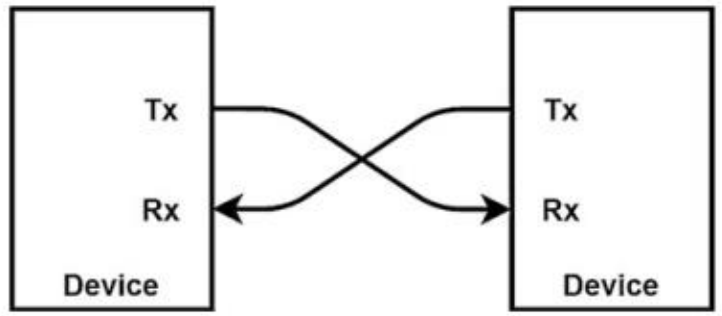
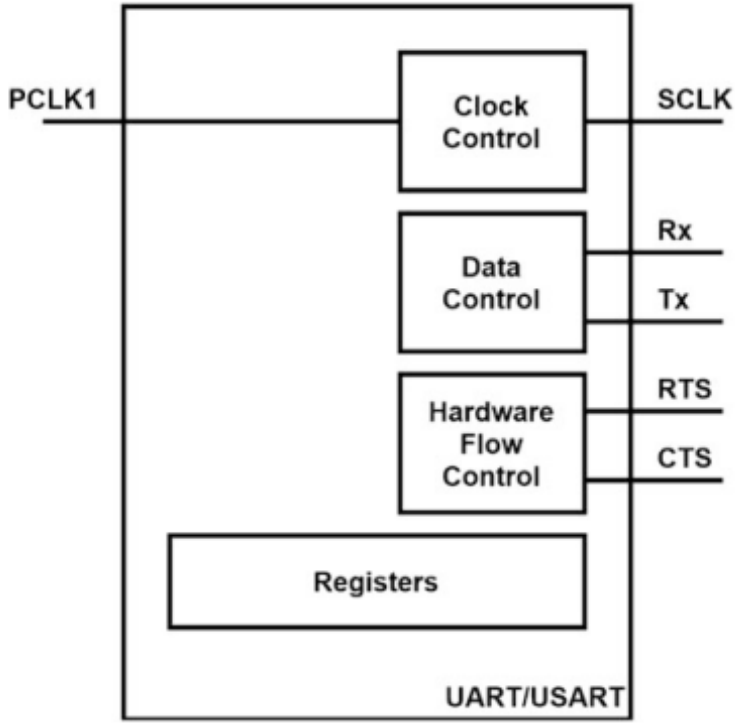


Fig. 8.5 General setup for UART communication between two devices

UART Modules in the STM32F4 Microcontroller

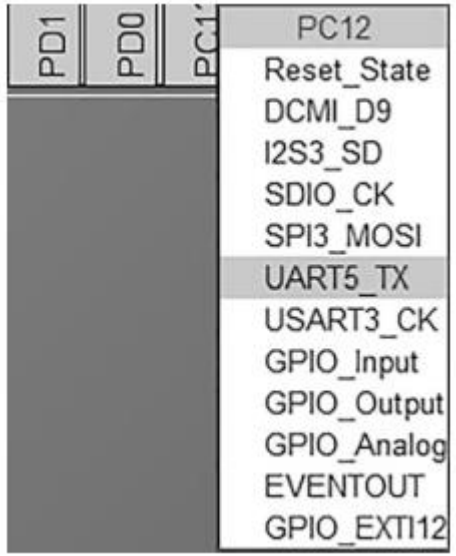
- The STM32F4 microcontroller has four UART modules called UART4, UART5, UART7, and UART8. The STM32F4 microcontroller also has four USART modules called USART1, USART2, USART3, and USART6. These modules can be used in UART mode.
- We provide block diagram of a generic USART/UART module for the STM32F4 microcontroller in Fig. 8.6. In this figure, the clock pin will not be used when UART is active. Here, the clock is provided by APB1 peripheral clock (PCLK1). There are two pins for data transfer as Tx and Rx. The module has hardware flow control support which requires two extra pins as request to send (RTS) and clear to send (CTS). UART does not need these pins. Finally, there is a clock output pin for USART.

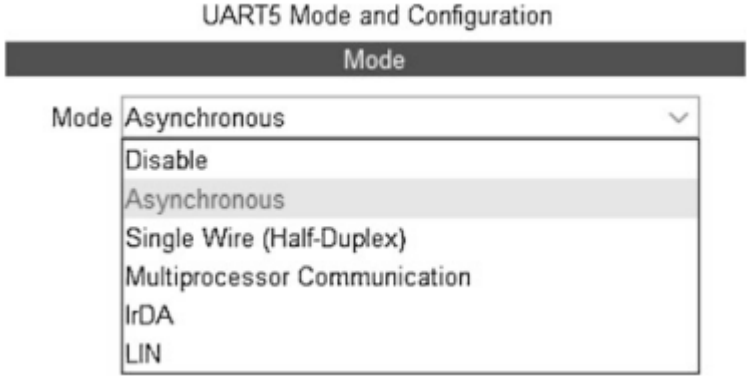
Fig. 8.6 USART/UART block diagram in the STM32F4 microcontroller

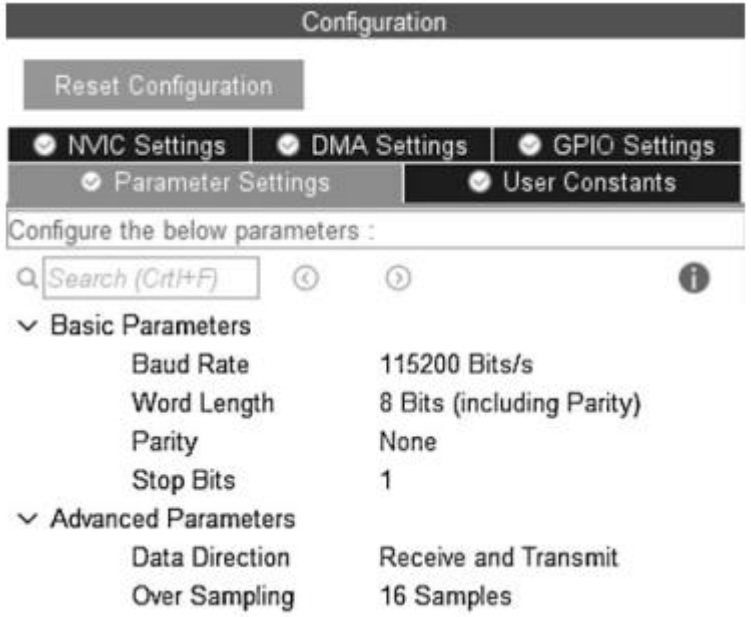


- **The USART1 module is connected to ST-LINK. Hence, it can communicate with PC through virtual COM port. As a side note, this port is generated by USB which is not an actual RS232 port. Through it, we can transfer data between the microcontroller and PC as if we are communicating with a real communication port.**
- The USART1 and USART6 can reach baud rates up to 11.25 Mbits/s. The other modules can reach a maximum baud rate of 5.625 Mbits/s. All supported baud rate values for UART and USART modules are given in the datasheet.
- The USART and UART modules in the STM32F4 microcontroller support IrDA and LIN protocols. IrDA is the infrared light-based communication method. The STM32F4 microcontroller only supports the IrDA SIR ENDEC method which uses the RZI data encoding. As a reminder, when data to be transmitted is logic level 0, an infrared light pulse is submitted. If data to be transmitted is logic level 1, nothing is sent.
- LIN is the low-cost protocol developed specifically for automotive industry. It is introduced since CAN bus is expensive. Therefore, it is used in noncritical applications such as car wiper, window, and ventilating control. LIN protocol works in half duplex mode. It supports connecting multiple slave devices to a single master device. The master device can issue a 6 bit message. All connected slave devices listen to this message. Only one slave responds to it and sends back a message to the master device in the form of 2 to 6 data bytes.
- All USART modules in the STM32F4 microcontroller support smartcard protocol used in communicating with chip cards. This protocol is similar to UART. The STM32F4 microcontroller supports the ISO 7816-3 standard for this purpose. According to this standard, each data packet is composed of 1 start bit, 8 data bits, 1 parity bit, and 1.5 stop bits. The 1.5 stop bits stand for sending the stop bit for 1.5 clock periods. The smartcard operation is half duplex such that only the Tx pin is used in communication.

UART Setup in the STM32F4 Microcontroller









- The UART module can operate in two modes as blocking (polling) or nonblocking (interrupt).
- In the blocking operation mode, the module waits (polling) until the transmit or receive operation has finished once it starts sending or receiving data in a loop. However, this mode blocks other routines while it waits. A timeout duration can be configured to terminate the operation if transmission or reception is not finished in due time.
- In the non-blocking operation mode, the module sends or receives data using first in, first out (FIFO) buffers. As the transmit or receive operation ends, an interrupt is generated. This operation does not block the code routine. That is why this mode is called non-blocking.

- If an interrupt is not required, then this step can be skipped. Otherwise, the UART interrupt should be enabled from the NVIC tab in the configuration menu as in Fig. 8.9. The interrupt priority can be changed under the “System Core - NVIC” section.
- To note here, it is suggested to lower the UART interrupt priority since the HAL system uses the SysTick interrupt to create precise 1 ms ticks for internal operations and the HAL_Delay function. The SysTick interrupt has the highest priority among peripheral units. Hence, assigning a lower priority to the UART interrupt will not affect it. Otherwise, precision of the HAL_Delay function may degrade.

Setup via C++ Language

- We can benefit from Mbed and available functions there to adjust the UART/USART module properties in C++ language. To do so, we should first adjust the Rx and Tx pins. This can be done by the class *BufferedSerial* or *UnbufferedSerial*. Usage of these classes is similar to the class *DigitalIn*.
- Therefore, we should supply Tx and Rx pin names to the class *BufferedSerial* or *UnbufferedSerial*. Optionally, we can provide the Baud Rate value to the classes. If this value is not provided, then the BufferedSerial or UnbufferedSerial class uses the default **Baud Rate value which is 9600 bits/s.**
- The difference between BufferedSerial and UnbufferedSerial classes is as follows.
- The BufferedSerial class uses an intermediary buffer to send and receive data to transmit multiple bytes.
- The UnbufferedSerial class does not use such a buffer to transmit or receive data. Therefore, it is recommended to use the BufferedSerial class in terms of CPU load as all buffering operations are handled by HAL library functions. The BufferedSerial class uses circular buffers with different size. This size can be changed in the “mbed.json” file using the parameters drivers.uart-serial-txbuf-size and drivers.uart-serial-rxbuf-size. The default buffer size here is 256 bytes.
- The BufferedSerial and UnbufferedSerial class parameters can be configured after initialization. The configurable parameters are baud rate, data length (7 or 8 bits), parity bit (odd, even, none), and stop bit (1 or 2 bits). Moreover, the operation mode can be selected as blocking or non-blocking. A callback function can be called whenever an interrupt occurs in non-blocking mode.

- Let’s provide examples on the usage of UART/USART modules via C language. The first example, with the C code given in Listing 8.1, aims to familiarize the reader with the setup and usage of the UART/USART module. Here, a string and counter value is sent to PC from the STM32F4 microcontroller via ST-LINK virtual COM port every 2 s. The green LED connected to pin PG13 of the STM32F4 board is toggled once data is transmitted. In this example, the UART module is configured in blocking mode. On the PC side, we should use an application, such as Tera Term, to receive data sent from the STM32F4 microcontroller.
- In order to execute the code in Listing 8.1, create an empty project. Then, configure the pins PA9 and PA10 as USART1_TX and USART1_RX, respectively. Afterward, enable the asynchronous mode of the USART1 module; set the baud rate to 115200 bits/s, data word length to 8, parity to none, stop bits to 1, data direction to transmit only, and oversampling to 16. As the project is compiled and run, the string will be transmitted to the host PC via UART and the green LED will be toggled. Do not forget to use the terminal program to observe the sent string in the host PC.

sting 8.1 Usage of the UART/USART module in blocking operation mo

```
* USER CODE BEGIN Includes */
include "stdio.h"
* USER CODE END Includes */

* USER CODE BEGIN PV */
int8_t serData[] = " STM32F4 UART PC Example\r\n";
int8_t decNum[6];
int16_t counter = 0;
* USER CODE END PV */

* Infinite loop */

'* USER CODE BEGIN WHILE */
hile (1)

printf(decNum, "%d", counter++);
AL_UART_Transmit(&huart1, decNum, (uint16_t)sizeo
   1000);
AL_UART_Transmit(&huart1, serData, (uint16_t)size
   1000);
AL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
AL_Delay(2000);
'* USER CODE END WHILE */

'* USER CODE BEGIN 3 */

'* USER CODE END 3 */
```

Listing 8.2 Usage of the UART/USART module in non-blocking operation mode, the C code

```
/* USER CODE BEGIN PV */
uint8_t serRxData;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
if (serRxData == 'g')
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
else if (serRxData == 'r')
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);

__NOP();
HAL_UART_Receive_IT(&huart1, &serRxData, 1);
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
HAL_UART_Receive_IT(&huart1, &serRxData, 1);
/* USER CODE END 2 */
```

- The second example given in Listing 8.2 repeats the first example given in Listing 8.1 in non-blocking mode. This time when the character ’g’ is received, the green LED connected to pin PG13 of the STM32F4 board is toggled. When the character ’r’ is received, the red LED connected to pin PG14 of the STM32F4 board is toggled. For execution, replace the code given in Listing 8.1 with the one in Listing 8.2. Then, enable the USART1 interrupt. As the project is compiled and run, the code will wait to receive data from the UART module. To test the code, send the characters ’r’ and ’g’ from the host PC to the STM32F4 microcontroller. Hence, we can observe that the red and green LEDs toggle.

- Our first example, with the C++ code given in Listing 8.4, sends data from the STM32F4 microcontroller to the host PC. To execute this code in Mbed Studio, we should copy and paste it to the “main.cpp” file in our active project. As can be seen in Listing 8.4, the baud rate is set to 115200 bits/s first. Then, the code sends a fixed string and counter value to the host PC and sleeps for 2 s in an infinite loop.

Listing 8.4 First example on the usage of the UART/USART module, the C++ code

```
#include "mbed.h"

#define WAIT_TIME_MS 2000

DigitalOut greenLED(LED1);
static BufferedSerial serialPort(USBTX, USBRX) ;

uint8_t serData[] = "STM32F4 UART PC Example\n\r";

int main(void)
{
serialPort.set_baud(115200);

while (1) {
greenLED = !greenLED;
serialPort.write(serData, sizeof(serData));
thread_sleep_for(WAIT_TIME_MS);
}
}
```

- The second example, with the C++ code given in Listing 8.5, repeats the example given in Listing 8.1. This time when the character ’g’ is received, the green LED connected to pin PG13 of the STM32F4 board is toggled. When the character ’r’ is received, the red LED connected to pin PG14 of the STM32F4 board is toggled. To execute this code in Mbed Studio, we should copy and paste it to the “main.cpp” file in our active project.

Listing 8.5 Second example on the usage of the UART/USART module, the C++ code

```
#include "mbed.h"

DigitalOut greenLED(LED1);
DigitalOut redLED(LED2);

static UnbufferedSerial serialPort(USBTX, USBRX);
uint8_t serRxData;

void UARTRxISR()
{
serialPort.read(&serRxData, 1);
if (serRxData == 'g')
greenLED = !greenLED;
else if (serRxData == 'r')
redLED = !redLED;
}

int main()
{
serialPort.set_blocking(false);
serialPort.attach(&UARTRxISR);
__enable_irq();

while (true);
}
```

- The third example, with the C++ code given in Listing 8.6, implements the loopback property of the UART module such that the transmitted data is received by the same microcontroller. Hence, the program flow can be debugged without additional hardware. To do so, connect the pin PC12 to PD2 of the STM32F4 board using a jumper cable. Then, copy and paste the code given in Listing 8.6 to the “main.cpp” file in the active project and run it. Here, UnbufferedSerial class is used to generate a UART receive interrupt. When the onboard button (connected to pin PA0) is pressed, a counter value is sent. Once this value is received, the callback function is executed.

```
#include "mbed.h"

DigitalOut greenLED(LED1);
DigitalOut redLED(LED2);
InterruptIn userButton(BUTTON1);

static UnbufferedSerial serialPort(PC_12, PD_2, 9600);

uint8_t serTxData = 0;
uint8_t serRxData;

void getButtons()
{
serialPort.write(&serTxData, 1);
serTxData++;
greenLED = !greenLED;
}

void UARTRxISR()
{
redLED = !redLED;
serialPort.read(&serRxData, 1);
}

int main()
{

serialPort.set_blocking(false);
serialPort.attach(&UARTRxISR);
userButton.fall(&getButtons);
__enable_irq();

while (true);
}
```